

Γραφικά Υπολογιστών και Συστήματα Αλληλεπίδρασης
Ακαδημαϊκό Έτος 2022-2023

ΠΡΟΓΡΑΜΜΑΤΙΣΤΙΚΗ ΑΣΚΗΣΗ 1-B

Πρατικάκης Εμμανουήλ 4780

Δημητρέσης Χρήστος 4351

18/11/2022

1. Περιγραφή της εργασίας και της υλοποίησης

Σε αυτή την προγραμματιστική άσκηση εξοικειωθήκαμε με τις συντεταγμένες του επιπέδου και με την λειτουργία της κάμερας.

Μας ζητήθηκε να σχεδιάσουμε ένα 3D ισοσκελές τραπεζοειδές πρίσμα και να υλοποιήσουμε την λειτουργία μιας κάμερας

Έχοντας από την προηγούμενη εργασία τον τίτλο, προσθέσαμε επιπλέον utf-8 χαρακτήρες για να καταλήξω στο επιθυμητό αποτέλεσμα (Εργασία 1B – Τραπεζοειδές Πρίσμα).

Τέλος για το πρώτο βήμα, στην συνθήκη του do-while loop τροποποιήσαμε στην συνάρτηση `glfwGetKey` το `GLFW_KEY_ESCAPE` με το `GLFW_KEY_BACK`.

Τώρα το loop ή αλλιώς το πρόγραμμα τερματίζει πατώντας το space button (καθώς και με το escape button).

ΣΧΕΔΙΑΣΜΟΣ 3Δ ΑΝΤΙΚΕΙΜΕΝΟΥ

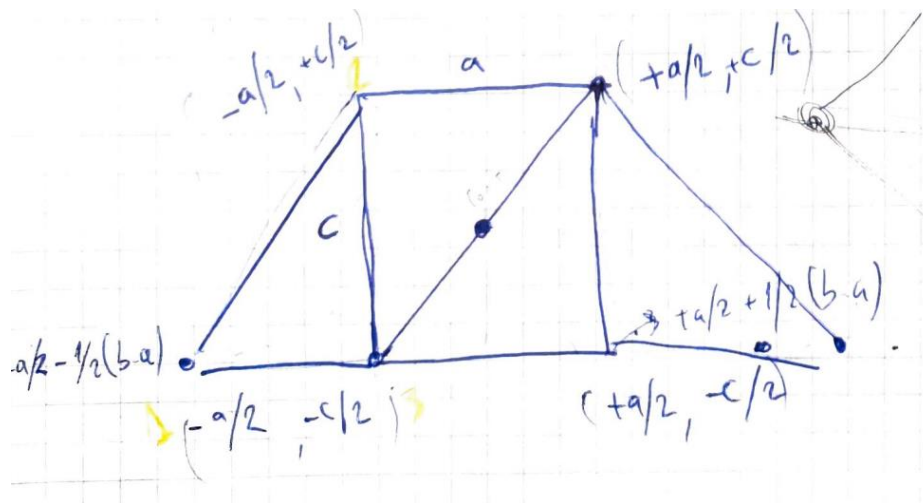
Για την ζωγραφική του 3D αντικειμένου ζητήθηκε από την άσκηση να έχει σαν κέντρο το σημείο (0,0,0) του επιπέδου.

Για τον σχεδιασμό του πρίσματος χρησιμοποίησα την λειτουργία της `glDrawArrays` το `GL_TRIANGLE_FAN`.

Στο VAO (Vertex Array Object) πέρασα τις συντεταγμένες οι οποίες ήταν 4 για κάθε πλευρά λόγω του mode στην `glDrawArrays`. Ωστόσο αν και η σχεδίαση των σχημάτων έγιναν σαν πολύπλευρα, τις συντεταγμένες τις βρήκαμε με την βοήθεια των τριγώνων.

Πιο αναλυτικά μας δίνεται η πληροφορία ότι από το κέντρο τραπεζίου περνάει ο άξονας z παράλληλα.

Στη παρακάτω εικόνα φαίνεται αναλυτικά.



Επίσης γνωρίζουμε ότι το κέντρο του τραπεζίου βρίσκεται στη μέση κατά μήκος και κατά πλάτος. Έτσι οι δύο βάσεις στον z άξονα θα απέχουν HEIGHT/2 και -HEIGHT/2 αντίστοιχα.

Το ύψος είναι διαφορετικό κάθε φορά που ανοίγει η εφαρμογή και προκύπτει μέσα από έναν Random Generator.

Για τα παραλληλόγραμμα απλά βασιστήκαμε στις συντεταγμένες που βρήκα για το τραπέζιο καθώς οι δύο βάσεις είναι παράλληλες ως προς το επίπεδο xy.

Για τον χρωματισμό των πλευρών περάσαμε τις κατάλληλες τιμές των χρωμάτων στο Color Vertex Array Object (g_color_buffer_data[]).

Πιο συγκεκριμένα χρησιμοποιήσαμε:

ΚΑΤΩ ΒΑΣΗ : ΚΟΚΚΙΝΟ

ΑΝΩ ΒΑΣΗ: ΚΙΤΡΙΝΟ

ΑΝΑΤΟΛΙΚΗ ΠΛΕΥΡΑ: ΓΑΛΑΖΙΟ

ΝΟΤΙΑ ΠΛΕΥΡΑ: ΕΛΑΦΡΥ ΚΥΑΝΟ (ΣΧΕΔΟΝ ΑΣΠΡΟ)

ΔΥΤΙΚΗ ΠΛΕΥΡΑ: ΠΡΑΣΙΝΟ

ΒΟΡΕΙΑ ΠΛΕΥΡΑ: ΦΟΥΞΙΑ

KΑΜΕΡΑ

Αρχικοποιήσαμε τους πίνακες ViewMatrix, ProjectionMatrix και ModeMatrix αντίστοιχα έξω από την `main()`.

Για τον ViewMatrix όρισα GLM διανύσματα μεγέθους 3 για την θέση της κάμερας, την κατεύθυνση της και το *up* της. Στο *up* στην ουσία βάζεις μονάδα στον άξονα που θες να κοιτάει προς τα θετικά. Στην δική μας περίπτωση βάλαμε στον άξονα z.

Και τέλος χρησιμοποιήσαμε συναρτήσεις τύπου Accessors για να παίρνω τον κάθε πίνακα ξεχωριστά μέσα στην `main()` όπου αυτός χρειάζεται.

Δημιουργήσαμε την συνάρτηση `computeMatricesFromInputs()` τύπου `void` για τις λειτουργίες του μοντέλου και της κάμερας, οι οποίες καθορίζονται από τα `input` του πληκτρολογίου.

1. SCALE UP/DOWN : <u> , <p>

Για την λειτουργία Scale Up/Down του πρίσματος χρησιμοποιήσαμε την συνάρτηση της βιβλιοθήκης GLM (και του extension της GTC), την `scale`. Επειδή θέλουμε μόνο να μεγαλώνει και να μικραίνει το ύψος του πρίσματος βάλαμε 1.0f τιμές για τον x και y άξονα ενώ για τον z βάλαμε τιμές μεγαλύτερες και μικρότερες του 1.0f για την κλιμάκωση και την σμίκρυνση αντίστοιχα.

2. ΚΙΝΗΣΗ ΚΑΜΕΡΑΣ ΓΥΡΩ ΑΠΟ ΤΟΝ ΑΞΟΝΑ X:

<w> , <x>

Για την λειτουργία περιστροφής της κάμερας γύρω από τον άξονα x χρησιμοποιήσαμε ένα extension της βιβλιοθήκης της GLM. Καθορίστηκε στην αρχή του κώδικα σαν `#include <glm/gtx/rotate_vector.hpp>`

Έτσι περιστρέψαμε το position της κάμερας (ViewMatrix). Για τη γωνία περιστροφής χρησιμοποιήσαμε την μεταβλητή που ορίσαμε παραπάνω σαν cameraSpeed με θετικές τιμές για την δεξιόστροφη περιστροφή και αρνητικές για την αντίθετη της.

3. ΚΙΝΗΣΗ ΚΑΜΕΡΑΣ ΓΥΡΩ ΑΠΟ ΤΟΝ ΑΞΟΝΑ X:

<q> , <z>

Για την λειτουργία περιστροφής της κάμερας γύρω από τον άξονα z εφαρμόσαμε ξάνα τον προαναφερόμενο τρόπο. Μόνο που αυτή τη φορά το position της κάμερας παίρνει νέες τιμές με την συνάρτηση rotateZ(). Αντίστοιχα και για αυτή την περιστροφή χρησιμοποιήσαμε σαν γωνία της μεταβλητή cameraSpeed.

4. ZOOM IN/OUT: <+> , <->

Για την λειτουργία του Zoom In και Zoom Out προσθαφρέσαμε το cameraSpeed που ουσιαστικά είναι το shutter speed της κάμερας από το FoV (Field of View – Πεδίο Όρασης). Το πεδίο όρασης λειτουργεί αντίστροφα από την μεγέθυνση. Δηλαδή όσο μεγαλώνει το FoV τόσο γίνεται σμίκρυνση στο αντικείμενο. Και αντίστροφα όσο μικραίνει το FoV, τόσο μεγενθύνεται το αντικείμενο.

Επομένως για το Zoom In έχουμε $FoV -= cameraSpeed$ και για το Zoom Out έχουμε $FoV += cameraSpeed$.

Η υλοποίηση έγινε σε υπολογιστή με Windows 10 x64 και το περιβάλλον που χρησιμοποιήσαμε ήταν το Visual Studio 2022.

2. Περιγραφή Δυσκολιών

Με βάση την παραπάνω εικόνα που εξηγήσαμε πως βρήκαμε συντεταγμένες για το 3Δ αντικείμενο, δοκιμάσαμε να χρησιμοποιήσουμε το GL_TRIANGLES mode της `glDrawArrays` και για κάποιο λόγο δεν εφάπτοναν τα τρίγωνα μεταξύ τους.

3. Αναφορές

OpenGL Documentation

<https://docs.gl/>

Ιδέα για τον τίτλο του προγράμματος με ελληνικούς χαρακτήρες

<https://stackoverflow.com/questions/58801165/cant-display-%C3%A4-in-glfw-window-title>

Ισοσκελές Τραπεζίο

<https://mathworld.wolfram.com/IsoscelesTrapezoid.html>

Πληροφορίες για την λειτουργία της GL_TRIANGLE_FAN

<https://stackoverflow.com/questions/20394727/gl-triangle-strip-vs-gl-triangle-fan>

Random Generator in C++

<https://www.digitalocean.com/community/tutorials/random-number-generator-c-plus-plus>

Coordinate Systems

<https://learnopengl.com/Getting-started/Coordinate-Systems>

Camera Explanation

<https://learnopengl.com/Getting-started/Camera>

GLM – GTC Matrix Transformation

<https://stackoverflow.com/questions/8844585/glm-rotate-usage-in-opengl>

FoV - Zoom

<https://www.agi.com/tech-tips/zoom-in-to-the-field-of-view>

Why no plus key in GLFW_KEY_PRESS

<https://github.com/glfw/glfw/issues/1494>